Day 16:

1. Launch a thread using a lambda function that prints the current thread ID.

Code:

```
#include <iostream>

#include <thread>

using namespace std;


int main() {

    thread t([]() {

        cout << "Current Thread ID: " << this_thread::get_id() << endl;

    });


    t.join();


    return 0;

}
```

Output:

```
Current Thread ID: 2

--------------------------------
Process exited after 1.809 seconds with return value 0
Press any key to continue . . .
```

2. Create two threads incrementing a shared counter without synchronization. Show incorrect results

Code:

```
#include <iostream>

#include <thread>

using namespace std;
```

```
int counter = 0;

void increment() {
    for (int i = 0; i < 100000; i++) {
        counter++;
    }
}

int main() {
    thread t1(increment);
    thread t2(increment);

    t1.join();
    t2.join();

    cout << "Final counter value: " << counter << endl;
    cout << "Expected value: 200000" << endl;

    return 0;
}
```

Output:

```
Final counter value: 200000
Expected value: 200000

--------------------------------
Process exited after 1.784 seconds with return value 0
Press any key to continue . . .
```

3.Use std::mutex to protect the shared counter and ensure correct output

Code:

```
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

int counter = 0;
mutex m;

void increment() {
    for (int i = 0; i < 100000; i++) {
        m.lock();
        counter++;
        m.unlock();
    }
}

int main() {
    thread t1(increment);
    thread t2(increment);

    t1.join();
    t2.join();
```

```
    cout << "Final counter value: " << counter << endl;
    cout << "Expected value: 200000" << endl;

    return 0;
}
```

Output:

```
Final counter value: 200000
Expected value: 200000

--------------------------------
Process exited after 1.696 seconds with return value 0
Press any key to continue . . .
```

4. Create a thread that waits for a signal and another thread that notifies it after 2 seconds.

Code:

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <chrono>
using namespace std;

mutex m;
condition_variable cv;
bool ready = false;

void waitingThread() {
    unique_lock<mutex> lock(m);
```

```
    cout << "Waiting for signal..." << endl;

    cv.wait(lock, [] { return ready; });

    cout << "Signal received! Thread is proceeding." << endl;
}

void notifyingThread() {
    this_thread::sleep_for(chrono::seconds(2));

    {
        lock_guard<mutex> lock(m);
        ready = true;
        cout << "Sending signal after 2 seconds..." << endl;
    }

    cv.notify_one();
}

int main() {
    thread t1(waitingThread);
    thread t2(notifyingThread);

    t1.join();
    t2.join();
```

```
    return 0;
}
```

Output:

```
Waiting for signal...
Sending signal after 2 seconds...
Signal received! Thread is proceeding.

--------------------------------
Process exited after 3.702 seconds with return value 0
Press any key to continue . . .
```

5. Write a program that uses std::this_thread::sleep_for() and std::this_thread::yield().

Code:

```
#include <iostream>

#include <thread>

#include <chrono>

using namespace std;


void task1() {

    cout << "Task 1 started" << endl;


    this_thread::sleep_for(chrono::seconds(2));
```

```
    cout << "Task 1 resumed after sleeping for 2 seconds" << endl;
}

void task2() {
    cout << "Task 2 started" << endl;

    this_thread::yield();

    cout << "Task 2 resumed after yielding" << endl;
}

int main() {
    thread t1(task1);
    thread t2(task2);

    t1.join();
    t2.join();

    return 0;
}
```

Output:

```
Task 1 startedTask 2 started

Task 2 resumed after yielding
Task 1 resumed after sleeping for 2 seconds

--------------------------------
Process exited after 3.703 seconds with return value 0
Press any key to continue . . .
```

6. Launch an asynchronous task to compute factorial and retrieve the result using std::future.

Code:

```
#include <iostream>

#include <future>

using namespace std;


int factorial(int n) {

    int fact = 1;

    for (int i = 1; i <= n; i++) {

        fact = fact * i;

    }

    return fact;

}


int main() {

    int num = 5;


    future<int> result = async(launch::async, factorial, num);


    cout << "Calculating factorial asynchronously..." << endl;
```

```
    cout << "Factorial of " << num << " is: " << result.get() << endl;


    return 0;
}
```

Output:

```
Calculating factorial asynchronously...
Factorial of 5 is: 120

--------------------------------
Process exited after 1.722 seconds with return value 0
Press any key to continue . . .
```

7.Write a simple program that performs heavy arithmetic in a loop.

Experiment: Compile with-00, -01, -02, and -03 (or /Od, /01, /O2 in MSVC).

Use the same program and compile with -Os (GCC/Clang) or /01 (MSVC).

Compare runtime and binary size against -02.

Code:

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;


int main() {
    volatile long long sum = 0;


    auto start = high_resolution_clock::now();


    for (long long i = 1; i <= 200000000; i++) {
```

```
        sum += (i * 3) / 2 + (i % 7) - (i % 5);
    }

    auto end = high_resolution_clock::now();

    auto duration = duration_cast<milliseconds>(end - start);

    cout << "Sum = " << sum << endl;
    cout << "Execution time = " << duration.count() << " ms" << endl;

    return 0;
}
```

Output:

```
Sum = 300000002999999998
Execution time = 432 ms

--------------------------------
Process exited after 2.135 seconds with return value 0
Press any key to continue . . .
```

8.Given n ropes and a target number of pieces k, find the maximum possible length to cut the ropes into at least k pieces using ternary search on the answer space.

Code:

```
#include <iostream>

#include <vector>

#include <iomanip>

using namespace std;
```

```
bool canCut(const vector<double>& ropes, double length, int k) {
    int pieces = 0;
    for (int i = 0; i < ropes.size(); i++) {
        pieces += (int)(ropes[i] / length);
    }
    return pieces >= k;
}

double maxRopeLength(vector<double>& ropes, int k) {
    double low = 0.0;
    double high = 0.0;

    for (int i = 0; i < ropes.size(); i++) {
        if (ropes[i] > high)
            high = ropes[i];
    }

    for (int iter = 0; iter < 100; iter++) {
        double mid1 = low + (high - low) / 3.0;
        double mid2 = high - (high - low) / 3.0;

        if (canCut(ropes, mid1, k))
            low = mid1;
        else
            high = mid2;
    }
```

```
    return low;
}

int main() {
    int n, k;
    cout << "Enter number of ropes: ";
    cin >> n;

    vector<double> ropes(n);
    cout << "Enter rope lengths:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> ropes[i];
    }

    cout << "Enter target number of pieces: ";
    cin >> k;

    double ans = maxRopeLength(ropes, k);

    cout << fixed << setprecision(6);
    cout << "Maximum possible length: " << ans << endl;

    return 0;
}
```

Output:

```
Enter number of ropes: 3
Enter rope lengths:
40
58
76
Enter target number of pieces: 80
Maximum possible length: 2.148148

--------------------------------
Process exited after 27.75 seconds with return value 0
Press any key to continue . . .
```

9.Given an array of integers and an integer k, find the maximum sum of any contiguous subarray of size k

Code:

```
#include <iostream>

using namespace std;


int main() {

    int n, k;


    cout << "Enter size of array: ";

    cin >> n;


    int arr[n];

    cout << "Enter array elements: ";

    for (int i = 0; i < n; i++) {

        cin >> arr[i];

    }


    cout << "Enter window size k: ";

    cin >> k;
```

```
    int windowSum = 0;

    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }

    int maxSum = windowSum;

    for (int i = k; i < n; i++) {
        windowSum = windowSum + arr[i] - arr[i - k];
        if (windowSum > maxSum) {
            maxSum = windowSum;
        }
    }

    cout << "Maximum sum of subarray of size " << k << " is: " << maxSum <<
endl;

    return 0;
}
```

Output:

```
Enter size of array: 5
Enter array elements: 2
5
8
10
26
Enter window size k: 3
Maximum sum of subarray of size 3 is: 44

--------------------------------
Process exited after 19.31 seconds with return value 0
Press any key to continue . . .
```

10.Given two strings s and t, find the smallest substring in s that contains all characters of t

Code:

```
#include <iostream>

#include <string>

#include <climits>

using namespace std;


int main() {

    string s, t;


    cout << "Enter string s: ";

    cin >> s;


    cout << "Enter string t: ";

    cin >> t;


    int need[256] = {0};

    int have[256] = {0};
```

```
for (int i = 0; i < t.length(); i++) {
    need[(unsigned char)t[i]]++;
}

int required = t.length();
int left = 0;
int minLen = INT_MAX;
int start = 0;

for (int right = 0; right < s.length(); right++) {
    char ch = s[right];

    if (need[(unsigned char)ch] > 0) {
        have[(unsigned char)ch]++;
        if (have[(unsigned char)ch] <= need[(unsigned char)ch]) {
            required--;
        }
    }

    while (required == 0) {
        if (right - left + 1 < minLen) {
            minLen = right - left + 1;
            start = left;
        }

        char leftChar = s[left];
```

```
            if (need[(unsigned char)leftChar] > 0) {
                have[(unsigned char)leftChar]--;
                if (have[(unsigned char)leftChar] < need[(unsigned char)leftChar]) {
                    required++;
                }
            }

            left++;
        }
    }

    if (minLen == INT_MAX) {
        cout << "No such substring exists" << endl;
    } else {
        cout << "Smallest substring: " << s.substr(start, minLen) << endl;
    }

    return 0;
}
```

Output:

```
Enter string s: ADOBECODEBANC
Enter string t: ABC
Smallest substring: BANC

--------------------------------
Process exited after 40.14 seconds with return value 0
Press any key to continue . . .
```

11.Given an array of positive integers and an integer k, find the length of the longest contiguous subarray whose sum is less than or equal to k.

Code:

```
#include <iostream>

using namespace std;


int main() {

    int n, k;


    cout << "Enter size of array: ";

    cin >> n;


    int arr[n];

    cout << "Enter array elements: ";

    for (int i = 0; i < n; i++) {

        cin >> arr[i];

    }


    cout << "Enter value of k: ";

    cin >> k;


    int left = 0;

    int sum = 0;

    int maxLen = 0;


    for (int right = 0; right < n; right++) {
```

```
        sum += arr[right];

        while (sum > k) {
            sum -= arr[left];
            left++;
        }

        int currentLen = right - left + 1;
        if (currentLen > maxLen) {
            maxLen = currentLen;
        }
    }

    cout << "Length of longest subarray: " << maxLen << endl;

    return 0;
}
```

Output:

```
Enter size of array: 5
Enter array elements: 2
3
5
7
12
Enter value of k: 4
Length of longest subarray: 1

--------------------------------
Process exited after 13.66 seconds with return value 0
Press any key to continue . . .
```

12. Given a string s and a pattern p, return all starting indices of p's anagrams in s.

Code:

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
    string s, p;
    cout << "Enter string s: ";
    cin >> s;
    cout << "Enter pattern p: ";
    cin >> p;

    vector<int> result;

    if (p.length() > s.length()) {
        cout << "No anagrams found" << endl;
        return 0;
    }

    int countP[26] = {0};
    int countS[26] = {0};

    for (int i = 0; i < p.length(); i++) {
        countP[p[i] - 'a']++;
        countS[s[i] - 'a']++;
```

```
}

if (equal(countP, countP + 26, countS)) {
    result.push_back(0);
}

for (int i = p.length(); i < s.length(); i++) {
    countS[s[i] - 'a']++;
    countS[s[i - p.length()] - 'a']--;

    if (equal(countP, countP + 26, countS)) {
        result.push_back(i - p.length() + 1);
    }
}

if (result.empty()) {
    cout << "No anagrams found" << endl;
} else {
    cout << "Starting indices of anagrams: ";
    for (int i = 0; i < result.size(); i++) {
        cout << result[i] << " ";
    }
    cout << endl;
}

return 0;
```

}

Output:

```
Enter string s: cbaebabacd
Enter pattern p: abc
Starting indices of anagrams: 0 6

--------------------------------
Process exited after 16.45 seconds with return value 0
Press any key to continue . . .
```